

Universidade Estadual de Mato Grosso do Sul

Ciência da Computação

Banco de Dados

Conceitos e arquitetura do SBD

Prof. Nilton
nilton@comp.uems.br

→ Modelagem

- ◆ Conceitos e arquitetura do SBD;
- ◆ Modelo de dados entidade-relacionamento – modelo ER;
- ◆ Modelo de dados relacional;
- ◆ Mapeamento ER para o relacional.

Modelos de Dados

→ Modelo de dados

- ◆ Descreve a estrutura de um banco de dados numa determinada etapa do projeto
- ◆ Oferece os meios para alcançar a abstração de dados

→ Classificação segundo o nível de abstração

- ◆ Modelo conceitual
- ◆ Modelo lógico
- ◆ Modelo físico

Modelo Conceitual

→ descreve a estrutura do BD de forma independente de qualquer implementação

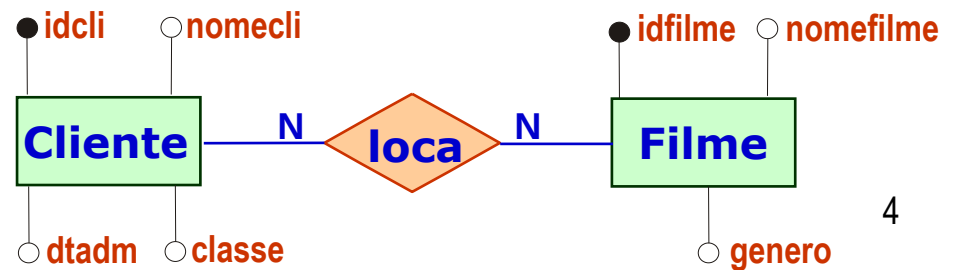
→ próximo como os usuários finais percebem os dados

→ exemplo

◆ Modelo Entidade-Relacionamento

◆ Diagrama E-R

→ notação de Peter Chen. 1976/The Entity-Relationship Model: Toward the unified view of data



Modelo Lógico

- descreve a estrutura do BD em função de uma futura implementação usando um SGBD
- oculta muitos detalhes do armazenamento de dados em disco
- fácil de compreensão de usuários finais
- derivado do modelo conceitual
- exemplo

◆ Modelo Relacional

- 📄 baseado em relações
- 📄 representação tabular (plana)

◆ Esquema relacional

Modelo lógico

Cliente(idcli, nomecli, dtadm, classe)

Filme(idfilme, nomefilme, genero)

Loca(idfilme, idcli, data)

Tabelas de um BD Relacional

Cliente

idcli	nomecli	dtadm	classe
1	Antonio	12/02/1970	A
2	Maria	20/05/1978	A
3	Francisco	01/05/1983	B
...	...	...	...

Filme

idfilme	nomefilme	genero
1	Dia D	10
2	O céu é o Limite	10
3	O Clone	50
...	...	...

Loca

idfilme	idcli	data
1	2	20/01/2004
3	2	20/01/2004
1	1	20/01/2004
...	...	...

Modelo Físico

→ descreve os detalhes de armazenamento (interno) dos dados como arquivos e das formas de acesso a esses dados

- ◆ Formato de registros, ordenações de registros, caminhos de acesso

→ físico

- ◆ para garantir a implementação possível

- ◆ primar pelo desempenho

→ derivado do modelo lógico

→ como?

- ◆ detalhes de armazenamento: organização de arquivos

- ◆ acesso aos dados: utilização de índices

Esquemas e Instâncias

→ Um banco de dados apresenta esquema e instância

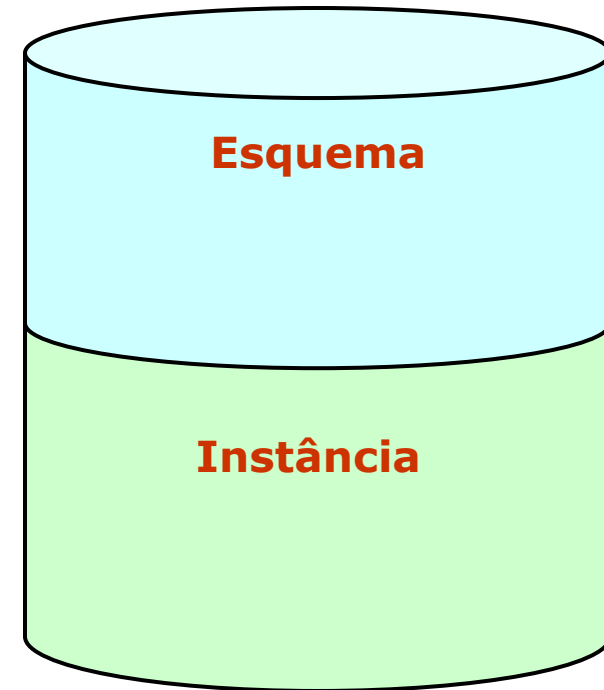
→ Esquema

- ◆ Definição/descrição do BD
- ◆ Estático, invariante no tempo
- ◆ Diagrama de esquema mostra alguns aspectos

→ Instância

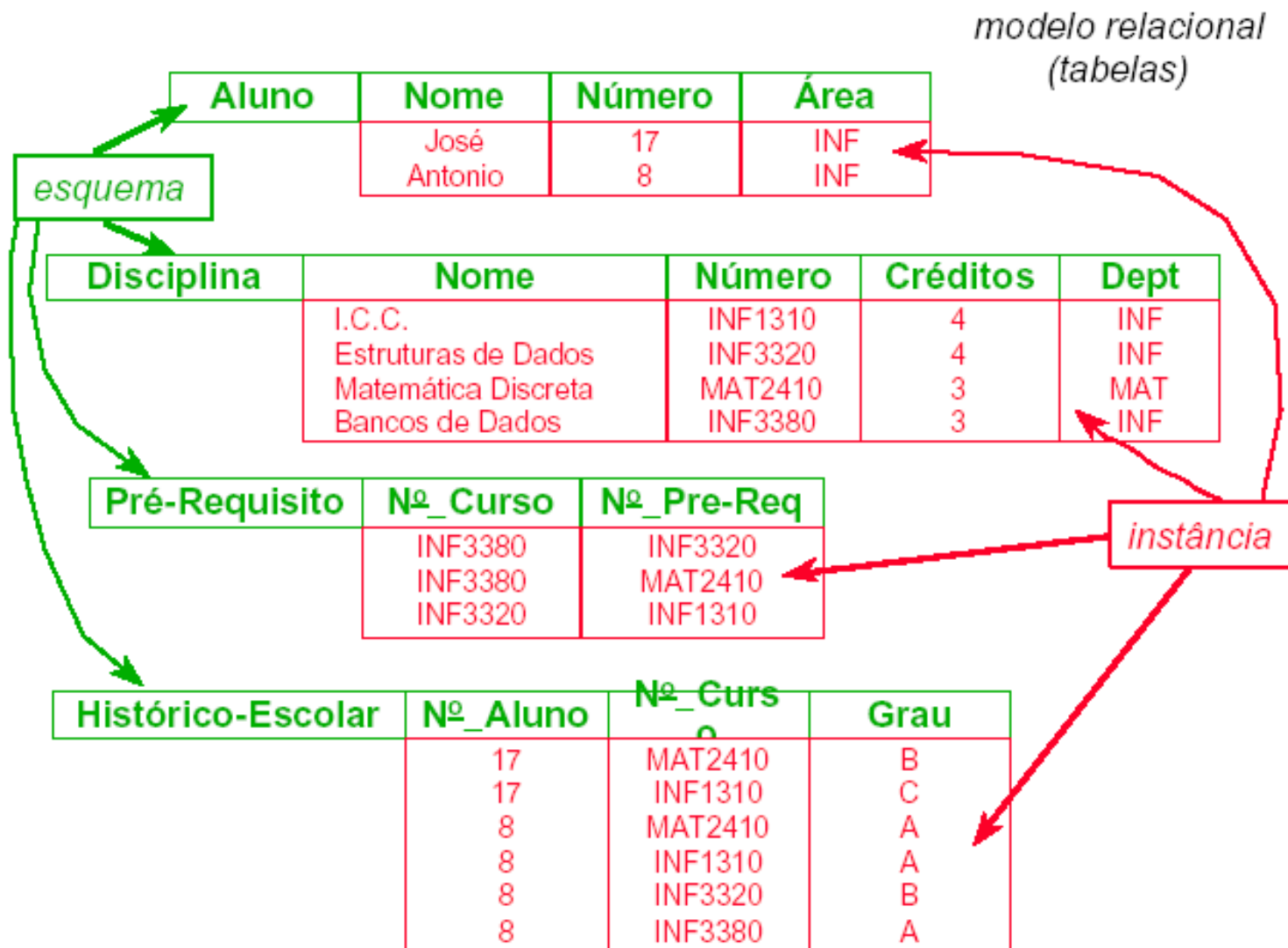
- ◆ Manipulação do BD
- ◆ Dinâmica, variante no tempo
- ◆ Determina um estado do BD

📄 Estados: vazio, inicial, atual



→ O SGBD deve garantir o estado do BD sempre válido

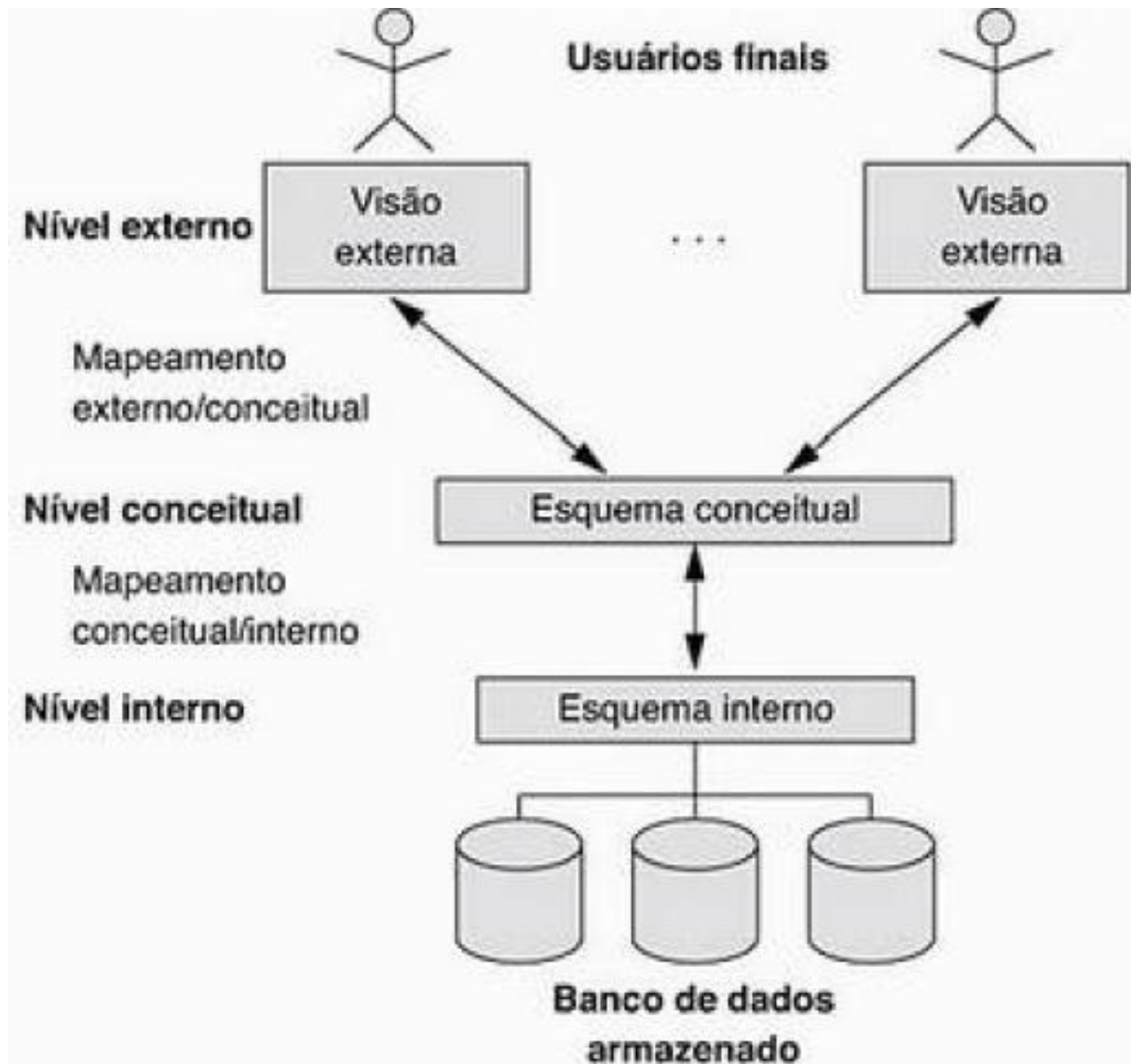
Esquemas e Instâncias



Arquitetura de 3 Esquemas

- Tem por objetivo separar as aplicações do usuário do banco de dados físico
- Foi proposta para ajudar a alcançar e visualizar as 3 características importantes da abordagem de BD
 1. uso de um catálogo para armazenar a descrição (esquema) do banco de dados de modo a torná-lo autodescritivo
 2. isolamento de programas e de dados
 3. Suporte para múltiplas visões
 4. Compartilhamento de dados e processamento de transações multiusuários (não);

Arquitetura de 3 Esquemas



Esquemas de Visão

- **incluiu esquemas externos ou visões do usuário**
- **cada esquema externo descreve a parte do banco de dados visível para um grupo de usuários**
- **uso de um modelo de dados de alto nível**

Esquema Conceitual

- descreve a estrutura do banco de dados inteiro para uma comunidade de usuários
 - ◆ Entidades, tipos de dados, relacionamentos, operações do usuário e restrições
- oculta detalhes de armazenamento físico e de acesso
- unificador das visões dos usuários
- uso de um modelo de dados de alto nível

Esquema Interno

- descreve a estrutura de armazenamento físico do banco de dados com objetivo de melhorar o desempenho
- caminhos de acesso
 - ◆ índices
 - ◆ Hashing
- uso de um modelo de dados físico

Arquitetura de 3 Esquemas

- A maioria dos SGBDs não separa os três níveis completo e explicitamente
 - ◆ Exige custo de mapeamento entre os níveis
- Sistemas antigos podem incluir detalhes de nível físicos no esquema conceitual
- A arquitetura ANSI de 3 níveis separa claramente o nível externo dos usuários, o nível conceitual do BD e o nível de armazenamento interno do projeto de um BD.

Independência de Dados

- A arquitetura de 3 esquemas pode ser utilizada para explicar o conceito de independência de dados
- É a capacidade de mudar o esquema num nível de um SBD sem ter que mudar o(s) esquema(s) no(s) nível(eis) seguinte(s) mais alto
- Independência lógica de dados
 - ◆ Alterar o esquema conceitual sem alterar esquema físico ou as aplicações
 - 📄 Expandir ou reduzir o banco de dados (adicionar/remover um tipo de registro ou item de dados)
 - 📄 Alterar restrições
- Independência física de dados
 - ◆ Alterar o esquema interno sem alterar esquema conceitual ou externo
 - 📄 Criar estruturas de acesso adicionais

Arquitetura de 3 esquemas ajuda a alcançar a independência dos dados, tanto física como a lógica, mas o MAPEAMENTO desses níveis requer trabalho adicional durante a execução de uma consulta. Por isto, poucos SGBDs implementam a arquitetura de 3 esquemas completamente.

Linguagens do SGBD

→ Linguagem de definição de dados

◆ *(DDL – Data Definition Language)*

→ Linguagem de definição do armazenamento

◆ *(SDL – Storage Definition Language)*

→ Linguagem de definição de visão

◆ *(VDL – View Definition Language)*

→ Linguagem de manipulação de dados

◆ *(DML – Data Manipulation Language)*

→ Linguagem hospedeira

DDL: data definition language

- definir os esquemas conceitual e interno quando não existe separação entre níveis conceitual e interno
- definir o esquema conceitual quando há a separação entre os níveis
- Uso pelo BDA ou projetista de BD

SDL: storage definition language

- **definir o esquema interno**
- **o mapeamento entre os dois esquemas (conceitual e interno) pode ser especificado por qualquer uma das linguagens (DDL, SDL)**
- **A maioria dos SGBDs relacionais não tem uma linguagem que implementa as funcionalidades da SDL**

VDL: view definition language

- especificar as visões dos usuários e seus mapeamentos para o esquema conceitual
- alguns SGBD usam DDL para especificar tanto o esquema conceitual como o externo

DML: data manipulation language

→ para incluir/alterar/modificar/consultar os dados

→ DML de alto nível ou não procedural

- ◆ utilizada sozinha em operações complexas do BD

- ◆ orientada a conjuntos de registros

 - 📄 Ex: SQL (select...from..where)

→ DML de baixo nível ou procedural

- ◆ embutida numa linguagem de programação

- ◆ orientada a registros (usa-se *loops*)

 - 📄 Ex: DL/1 (get unique, get next, ...)

Linguagem hospedeira

→ os comandos da DML estão embutidos na linguagem de programação

Nos SGBDs atuais as linguagens apresentadas não são consideradas distintas, é utilizada uma linguagem integrada e abrangente.

Ex: linguagem SQL (DDL+VDL+DML)

Interfaces do SGBD

→ Interfaces baseadas em menus

- ◆ Menus orientam a seleção dos dados num passo-a-passo (ex. menus *pull-down*)

→ Interfaces baseadas em formulários

- ◆ Entradas de um formulário para cada usuário são preenchidas nas operações sobre os dados (ex. Oracle Forms)

→ Interfaces gráficas (GUIs)

- ◆ Esquemas em formato de diagramas (uso de menus e formulários)

→ Interfaces de linguagem natural

- Diálogo

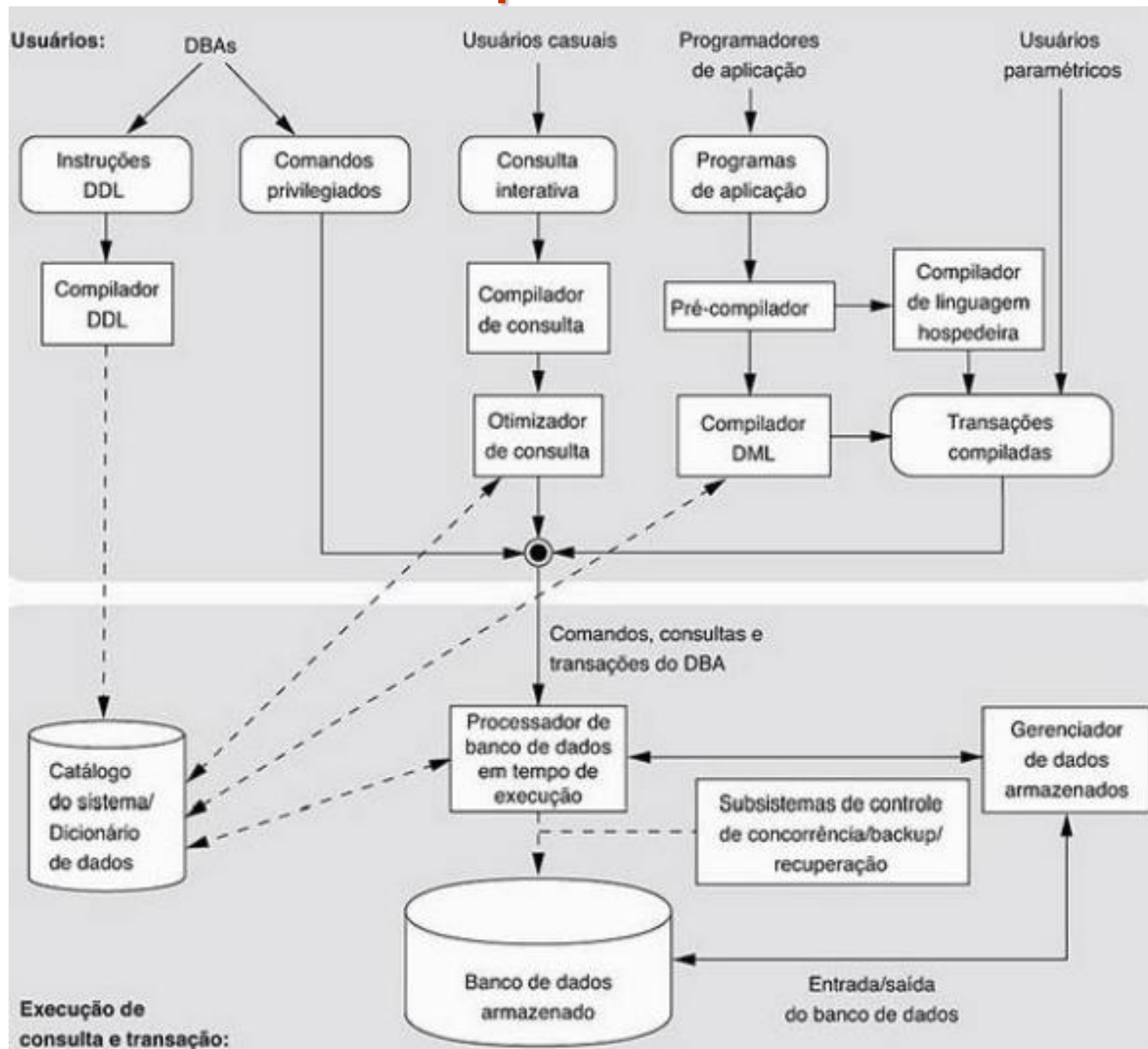
→ Interfaces para usuários paramétricos

- ◆ Conjunto pequeno de instruções de fácil acesso (teclas de função) projetado de acordo com tipo de usuário

→ Interfaces para DBAs

- ◆ Conjunto comandos privilegiados

Módulos Típicos de um SGBD



Módulos Típicos de um SGBD

→ **Compilador DDL**

- ◆ processa as definições de esquema e armazena as descrições de esquema no catálogo
- ◆ o catálogo inclui informações como nomes e tamanhos dos arquivos, nomes e tipos de dados dos itens de dados, detalhes de armazenamento de cada arquivo, informações de mapeamento entre esquemas, restrições, entre outras

→ **Processador de banco de dados em tempo de execução**

- ◆ Executa comandos privilegiados, os planos de consulta e as transações programadas
- ◆ Usa informações do catálogo e pode atualizar o catálogo com dados estatísticos
- ◆ Consome/produz dados armazenados através do gerenciador de dados armazenado
- ◆ Gerencia buffers na memória principal ou usa este recursos do SO

→ **Gerenciador de dados armazenados**

- ◆ Acessa os dados do banco de dados ou do catálogo
- ◆ Usa serviços básicos do SO para entrada/saída de baixo nível entre disco e memória principal

Módulos Típicos de um SGBD

→ **Compilador de consulta**

- ◆ Analisa e valida uma consulta transformando-a num formato interno

→ **Otimizador de consulta**

- ◆ Prepara um “ótimo” plano de execução de uma consulta baseado na reordenação das operações da consulta, na eliminação de duplicatas, no uso de algoritmos e índices eficientes
- ◆ Usa informações do catálogo

→ **Pré-compilador**

- ◆ Separa as instruções DML das instruções da aplicação de banco de dados e envia para os compiladores de DML e de linguagem hospedeira

→ **Compilador DML**

- ◆ Processa as instruções DML vindas da aplicação de banco de dados e as transforma em códigos objetos para o acesso ao banco de dados
- ◆ Usa informações do catálogo

Módulos Típicos de um SGBD

→ **Compilador de linguagem hospedeira**

- ◆ Processa as instruções da aplicação de banco de dados, gerando códigos objetos

→ **Transações compiladas**

- ◆ Constrói as transações programadas a partir dos códigos objetos gerados pelos compiladores de DML e de linguagem hospedeira
- ◆ As transações programadas inclui chamadas para o processador de banco de dados em tempo de execução

→ **Sistema de controle de concorrência/backup/recuperação**

- ◆ Integrado ao processador de banco de dados em tempo de execução

Utilitários de um SBD para um DBA

- Carga de arquivos
- Backup
- Reorganizador de arquivos
- Gerador de relatórios
- Monitor de desempenho
- Compactador de dados
- ...

Outras facilidades

- Ferramentas CASE
- Sistema de dicionário de dados
- Ambientes de desenvolvimento de aplicações
- Acesso remoto

Classificação dos SGBDs

→ Critério principal

1. Modelo de dados

📄 Relacional, Objeto, Objeto-Relacional, Hierárquico, Rede, XML

→ Outros critérios

2. Número de usuários

📄 Monousuário, Multiusuário

3. Número de locais

📄 Centralizado, Distribuído (homogêneo/heterogêneo)

4. Custo

📄 Opensource - código aberto (MySQL, PostgreSQL)

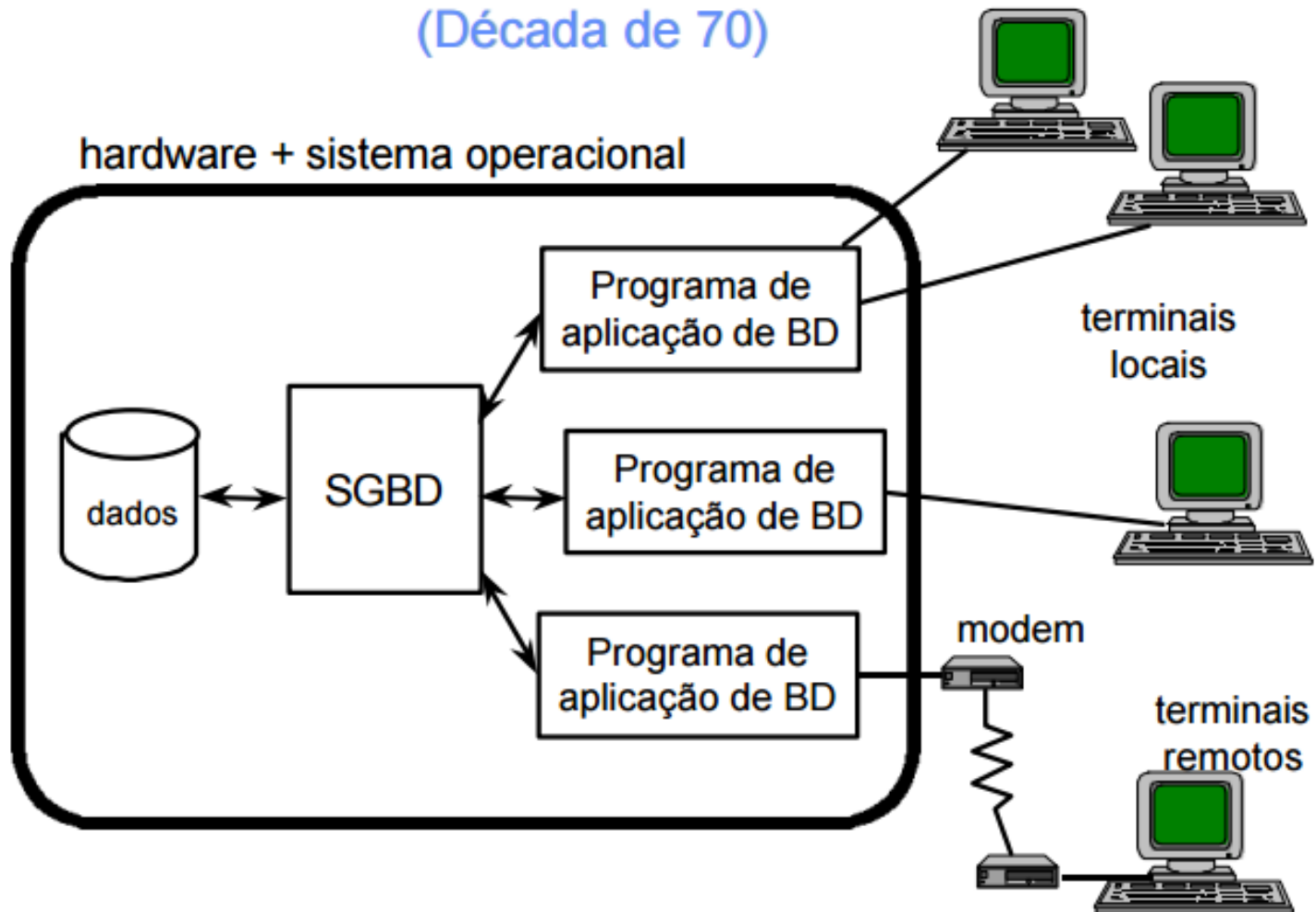
📄 Proprietário (Oracle, DB2, Server, Sybase)

5. Propósito

- Geral (sistemas transacionais)
- específicos (Geográficos, estatísticos, OLTP)

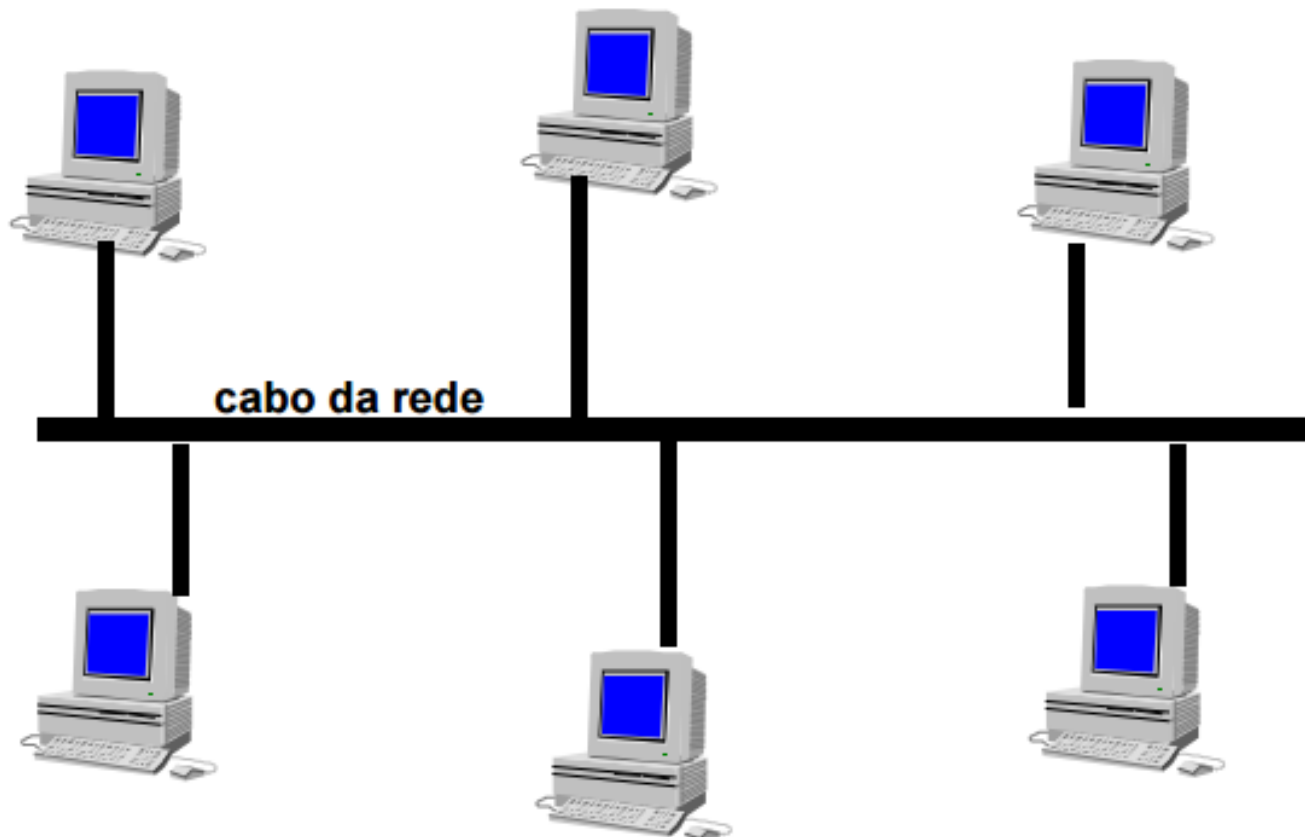
Bancos de Dados Centralizados

(Década de 70)



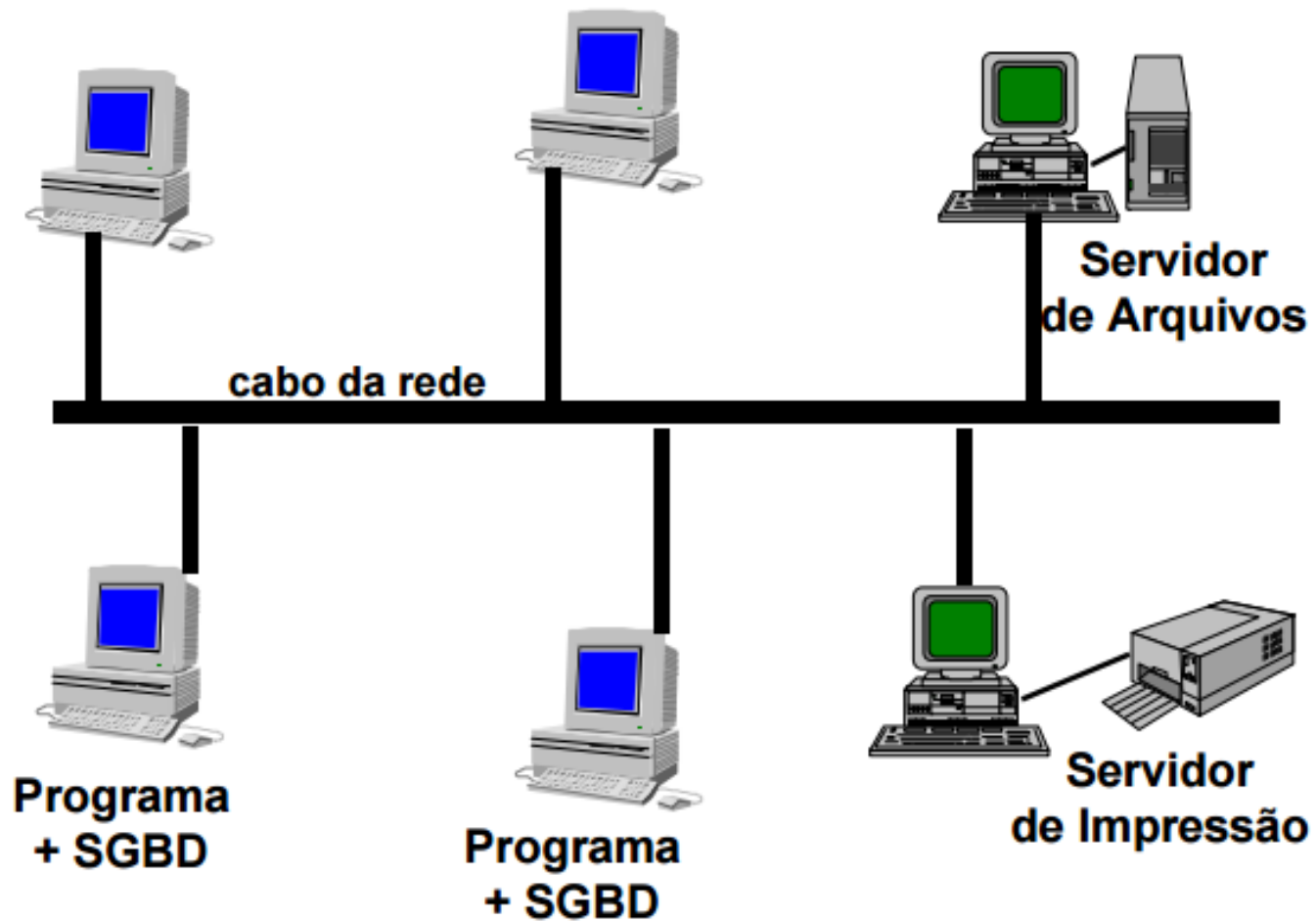
Redes de Microcomputadores

(Década de 80)

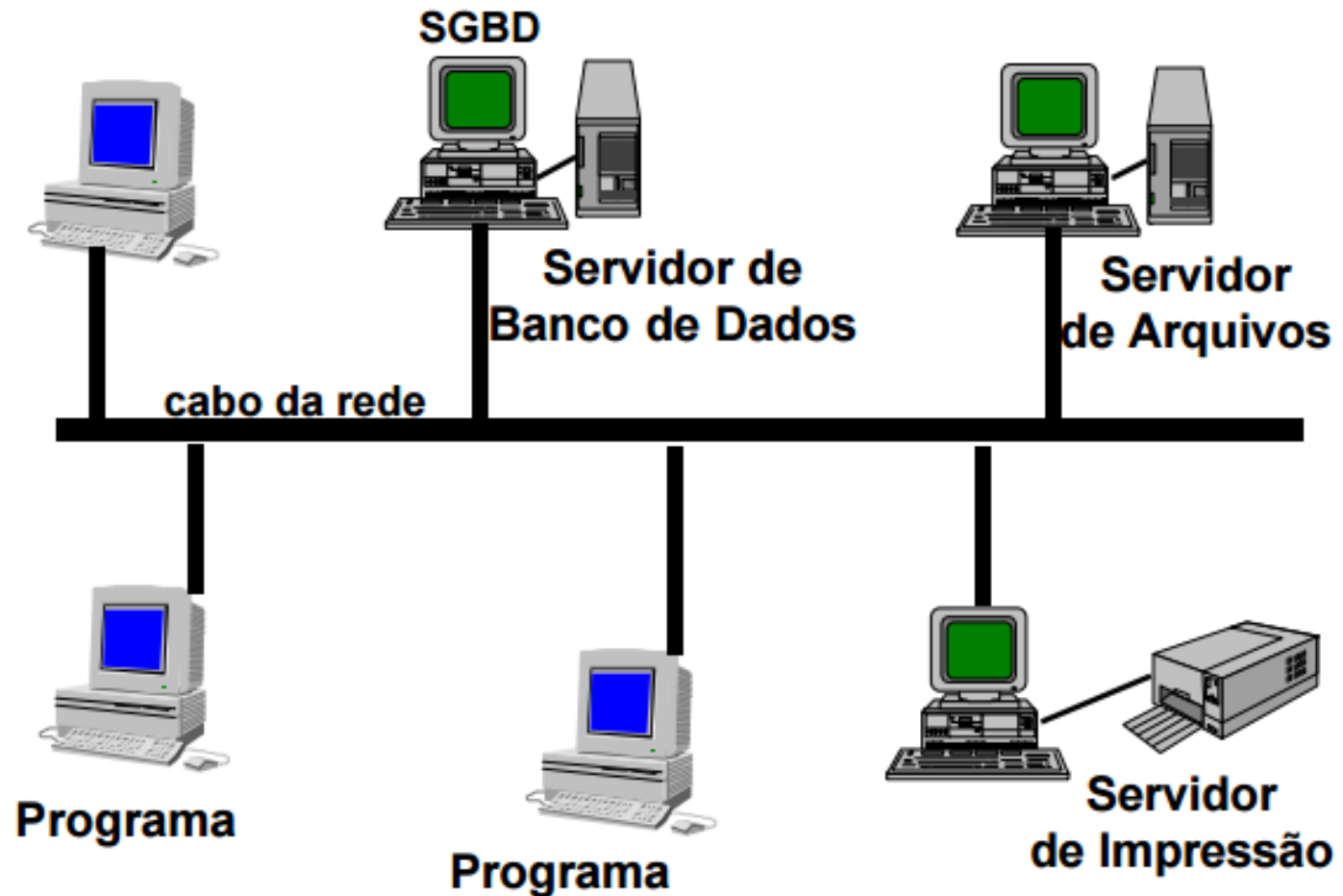


microcomputadores (estações de trabalho)
cada vez mais poderosos

Arquitetura Cliente/Servidor com Servidor de Arquivos

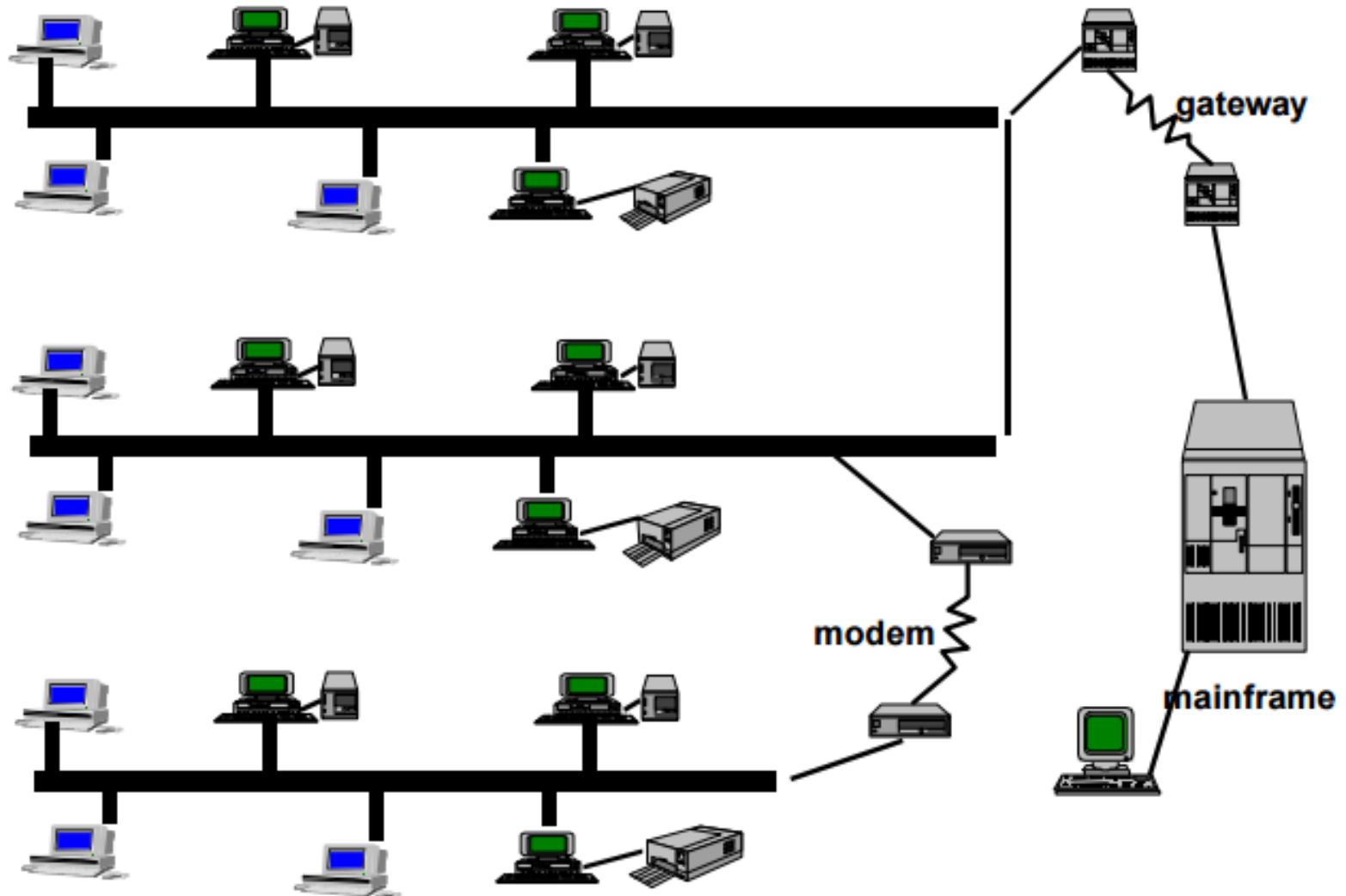


Arquitetura Cliente/Servidor com Servidor de Bancos de Dados (Arquitetura de 2 camadas)



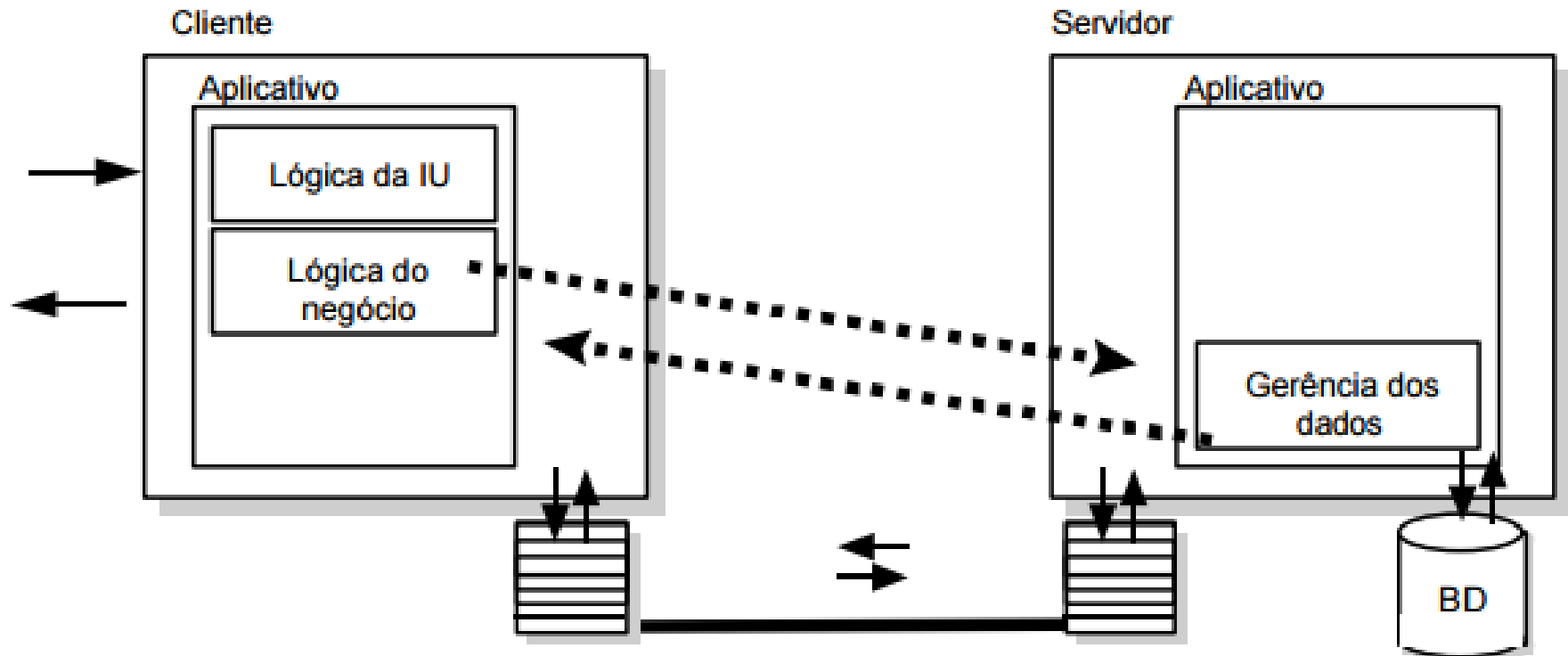
Arquitetura Cliente/Servidor com Múltiplos Servidores de Bancos de Dados Cooperativos

(Década de 90)

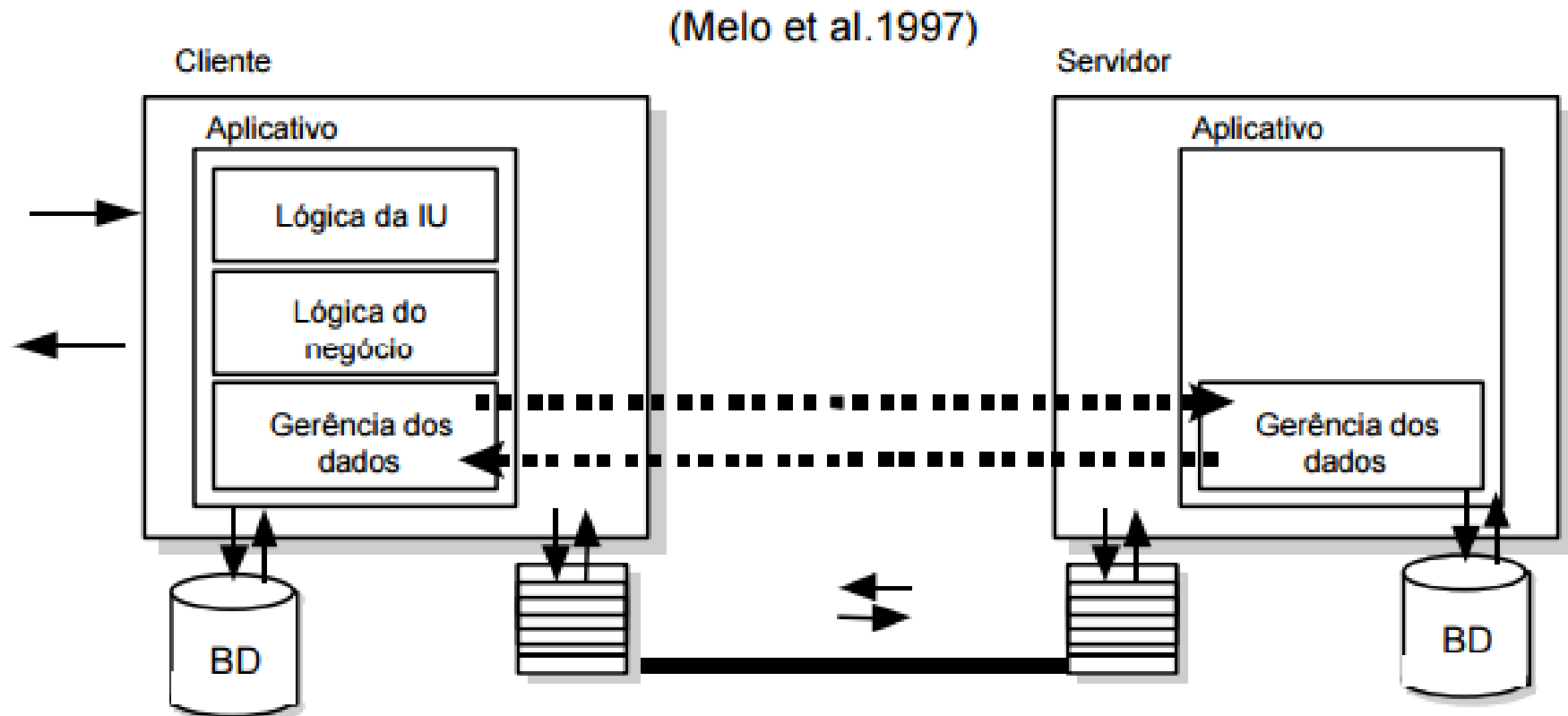


Banco de Dados Cliente/Servidor com Gerência de Dados Remota

(Melo et al.1997)

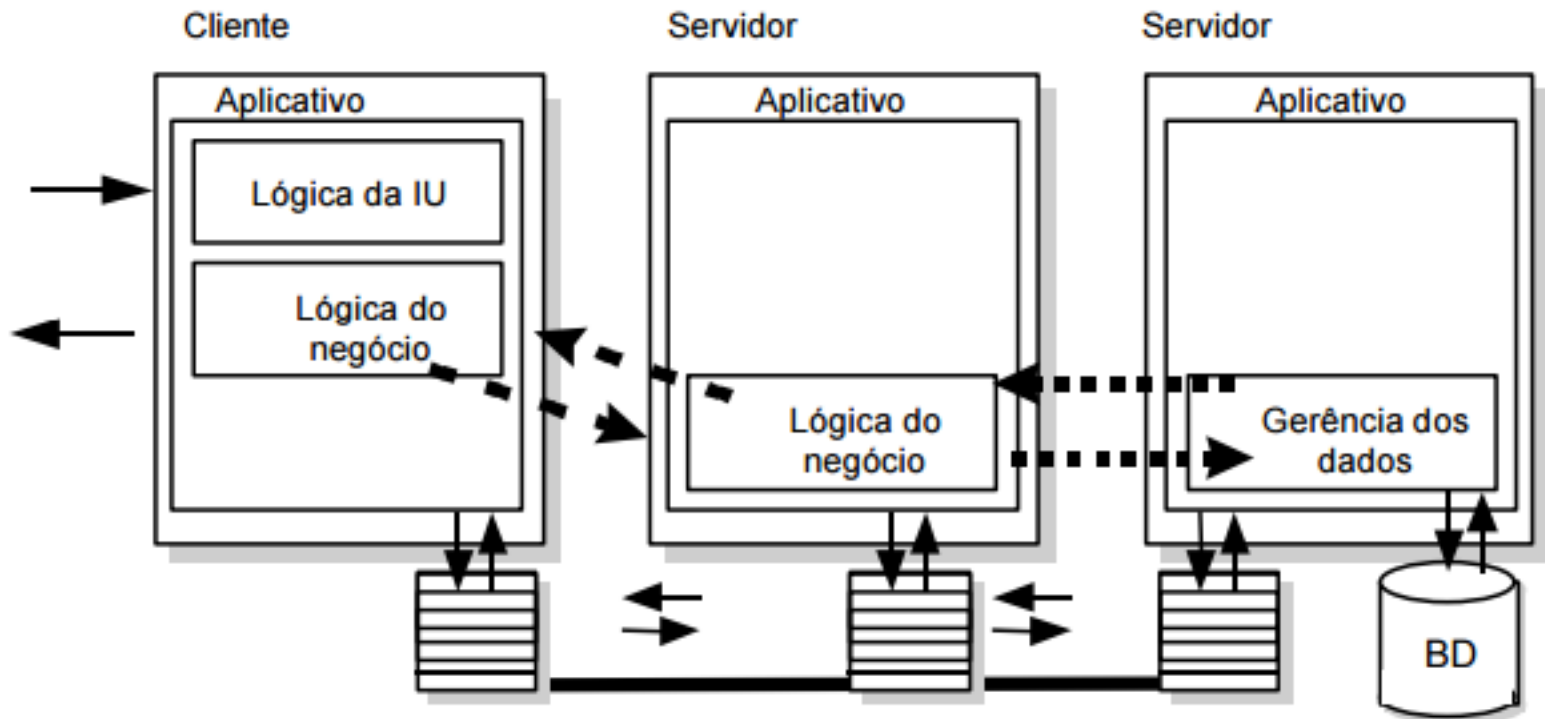


Banco de Dados Cliente/Servidor com Gerência de Dados Distribuída



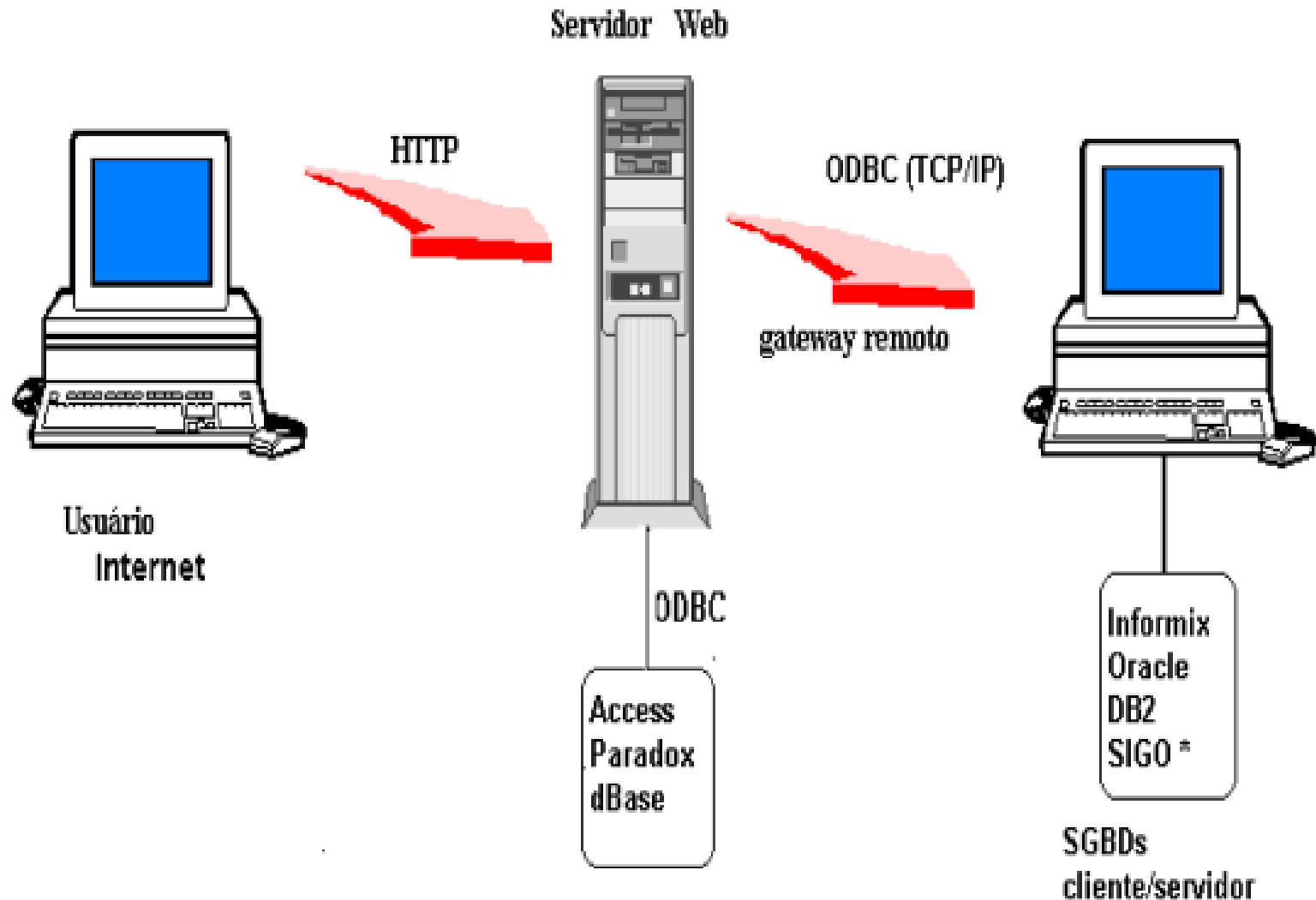
Banco de Dados Cliente/Servidor com três níveis (three-tier architecture)

(Melo et al.1997)



Programa de aplicação cliente-servidor com arquitetura em três níveis
No caso, as funções relacionadas com a lógica de negócio foram distribuídas entre a plataforma-cliente e uma plataforma servidora. O servidor de aplicação atua como cliente do servidor de banco de dados.

Bancos de Dados via aplicações Web (3 camadas)



Arquitetura de Intregração

